

LLD Learning Series - Day 1

Topics: Low-Level Design (LLD) and Object-Oriented Programming (OOP)

Lecture 1: What is Low-Level Design (LLD)?

The Big Picture

LLD is about designing the internal structure of an application. This means identifying classes, objects, their relationships, data flows, and where algorithm solutions fit in.

Concept	Role	Focus
DSA	The Brain	Algorithms: sorting, graphs, heaps, binary search
LLD	The Skeleton	Classes, objects, interactions, code organisation
HLD	The Architecture	Tech stack, databases, servers, scaling

Key Line to Remember

"If DSA is the brain, LLD is the skeleton of your application."

Story: Two Approaches to Building QuickRide (an Uber-like app)

Anurag's DSA-First Approach:

1. Problem decomposition:

- Map city intersections to graph nodes, roads to edges.
- Use Dijkstra's algorithm to compute shortest route.
- Use a min-heap (priority queue) to match riders to closest drivers.

2. Gaps:

- No identification of classes/entities (User, Rider, Location, Notification, Payment).
- Omits data security (masking phone numbers).
- Missing integration points (notifications, payment gateways).
- No consideration for scaling to millions of users.

Maurya's LLD-First Approach:

1. Entity identification:

Objects: User, Rider, Location, NotificationService, PaymentGateway, etc.

2. Define relationships and interactions:

- How User and Rider connect via Location.
- How NotificationService and PaymentGateway integrate.

3. Non-functional concerns:

Data security: Protect personal info.

Scalability: Architect code to handle millions of users without performance collapse.

4. Then apply DSA:

Embed shortest-path algorithm and driver-matching heap inside the object-oriented framework.

The 3 Core Goals of Good LLD

Scalability

Handle large user volumes. Code should allow adding servers or features with minimal effort.

Maintainability

New features must not break existing ones. Bugs should be easy to locate and fix.

Reusability

Write loosely coupled, plug-and-play modules reusable across different apps. For example, a notification module working for both Zomato and Amazon.

What LLD is NOT (vs HLD)

High-Level Design (HLD) is about system architecture, not code structure. It covers tech stack choices (e.g. Java Spring Boot), database decisions (SQL vs NoSQL), server scaling, load balancers, and cloud cost optimisation. LLD lives below all of that and is purely about how the code itself is structured.

Lecture 2: OOP - Abstraction and Encapsulation

Why Did We Move Beyond Procedural Programming?

Era	What Changed	Key Limitation
Machine Language	Raw 0s and 1s to the CPU	One bit flip breaks everything. Zero abstraction.
Assembly Language	Mnemonics instead of bits (e.g. MOV A, 61h)	Still tied to hardware. Hard to scale.
Procedural	Functions, if/else, loops, blocks	Poor real-world mapping. No data security. Cannot scale complex systems.
OOP	Objects model real-world entities	The solution: natural, secure, scalable.

Objects, Classes and Instances

Object: A real-world thing with attributes and behaviours.

Class: The blueprint defining those attributes (fields) and behaviours (methods).

Instance: A concrete object created in memory from that blueprint.

Pillar 1: Abstraction

Definition: Abstraction hides unnecessary implementation details and exposes only what is essential for the user of an object.

Real-world analogies:

Driving a car: You press pedals and turn the wheel. You do not need to know how fuel injection or gear synchronisation works. The car's interface abstracts all that.

Using a TV remote: Press a button, the channel changes. The signal encoding and infrared protocol are all hidden.

Using if/for/while: These keywords abstract away jump addresses and machine code. The compiler handles translation behind the scenes.

In Code (C++ example):

```
class Car {  
    virtual void startEngine() = 0; // pure virtual  
    virtual void accelerate() = 0;  
    virtual void brake() = 0;  
};
```

The Car class declares what operations must exist but hides how they work. Clients use Car pointers without needing concrete details. Multiple classes like SportsCar, SUV, and ElectricCar can all implement this same interface.

Benefits of Abstraction:

Simplified interfaces: clients focus on what an object does, not how.

Easy maintenance: internal changes do not affect client code.

Code reuse: many concrete classes share one abstract interface.

Reduced complexity: large systems broken into understandable modules.

Pillar 2: Encapsulation

Definition: Encapsulation bundles an object's data and the methods that operate on that data into one unit, and controls access to the internals.

The two facets of encapsulation:

Logical Grouping: All data and behaviour that belong together live in the same class. For example, a Car class holds engineOn, currentSpeed, shiftGear(), accelerate() all in one place.

Data Security: Restrict direct external access to prevent invalid operations. You can read the car's odometer but you cannot set it back to zero.

Real-world analogies:

A medicine capsule holds the medicine inside a protective shell. You swallow it without touching the contents directly.

A car odometer: visible from the dashboard but not tamperable.

Access Modifiers in C++:

Modifier	Who Can Access It
public	Everyone, anywhere
private	Only inside the class itself
protected	The class and its subclasses (used in inheritance)

Getters and Setters:

These are not just syntax. They are the gatekeepers of your object's integrity. Instead of exposing a field directly, you write a setter that validates the input before accepting it.

Benefits of Encapsulation:

Robustness: prevents accidental or malicious misuse of internal state.

Maintainability: internal changes do not ripple into client code.

Clear Contracts: clients interact only via well-defined public methods.

Modularity: self-contained units are easy to test and reuse.

My Takeaways from Today

1. LLD is not about writing smart algorithms. It is about designing how the whole system is organised before writing a single line of algorithm code.
2. Always identify your entities first (like Maurya did), then figure out how they talk to each other, then plug the algorithms in.
3. Abstraction means showing only the what. Encapsulation means protecting the how.
4. Getters and setters are not just syntax. They are the gatekeepers of your object's integrity.
5. Every time I use an if-statement or a for-loop, I am already benefiting from abstraction. The compiler does the heavy lifting.
6. The progression from Machine Language to Assembly to Procedural to OOP is the story of humans trying to think more naturally in code.